PicoMite Bluetooth Versions

PicoMiteBT is a build variant of PicoMite MMBasic that replaces the USB CDC console with a Bluetooth Low Energy serial link. Instead of plugging a USB cable in for the BASIC prompt, you connect wirelessly from a phone, laptop, or desktop computer.

Internally it advertises a Nordic UART Service (NUS) over BLE -- the same GATT profile used by Adafruit Bluefruit, Nordic dev kits, and most other BLE-serial peripherals -- so the device works out of the box with any BLE-NUS-aware terminal app, and with a small Python bridge for OSes that don't have one (Windows, Linux).

This document covers all three Bluetooth builds. Most of it is about PicoMiteBT, the console over Bluetooth. The two Bluetooth host builds, PicoMiteRP2350BTH and PicoMiteHDMIBTH, which take Bluetooth keyboards, mice and speakers instead, are covered in Bluetooth host versions near the end.

What you need

- A Raspberry Pi Pico 2 W (the pico2_w board -- Pico 2 alone does not have a wireless chip).
- The PICOBTRP2350 firmware variant -- built by BuildPicoMite.bat alongside the standard set, or directly with CMake: cmake -DCOMPILE=PICOBTRP2350 -G "NMake Makefiles" ..
- A BLE-capable host: any modern phone, almost any Windows 10+ laptop, any Linux machine with BlueZ 5.x, any modern macOS.
- For desktop use: Python 3.8+ with bleak (pip install bleak). The repo ships two scripts:
- ble_term.py -- recommended. A self-contained cross-platform VT100 terminal that talks directly to BLE NUS, with built-in XMODEM send/receive. No external terminal needed.
- ble_bridge.py -- alternative. BLE-to-TCP bridge for users who'd rather use their preferred terminal (Tera Term on Windows, PuTTY on Linux, etc.).

Device name

On startup the firmware builds a name from the last four hex characters of the chip's unique board ID, e.g. PicoMite-77BA. This name appears in every scan, and is what you point your tools at.

You can confirm the name on your specific board by scanning from a phone the first time.

Phone (Android / iOS)

Phones are the easiest path -- no bridge, no Python.

Android:

1. Install Serial Bluetooth Terminal (Kai Morich) from the Play Store.
2. Open the app, tap the menu -> Devices -> Bluetooth LE.
3. Tap the scan icon. Pick PicoMite-XXXX from the list.
4. The app will prompt you to pair -- accept (no PIN, "Just Works" pairing).
5. Back in the terminal view, you should see the MMBasic banner. Press Enter for the > prompt.

iOS:

1. Install Bluefruit Connect (Adafruit) -- free.
2. Connect to the PicoMite-XXXX device.
3. Use the UART tab. iOS handles the pairing transparently.

Subsequent re-connects don't require re-pairing -- the phone remembers the bond.

PicoMite Bluetooth Versions

Desktop (Windows / Linux / macOS) -- ble_term

ble_term.py is the recommended desktop path: one script, no external terminal, works identically on all three OSes. It talks directly to the Pico's BLE NUS service, puts the host console into raw + virtual terminal mode, and forwards bytes both ways. The host terminal renders the VT100 escape sequences MMBasic emits natively.

Setup

1. Install bleak:

```
pip install bleak
```

(pip3 install bleak on macOS.)

2. Pair the device once through OS settings -- same as for the bridge path:

- Windows: Settings -> Bluetooth & devices -> Add device -> Bluetooth -> pick PicoMite-XXXX.
- Linux:

```
bluetoothctl
[bluetooth]# scan on
[bluetooth]# pair 28:CD:C1:FF:B7:8D
[bluetooth]# trust 28:CD:C1:FF:B7:8D
[bluetooth]# exit
```

(you may need to be in the bluetooth group: sudo usermod -aG bluetooth $USER)

- macOS: System Settings -> Bluetooth -> click "Connect" next to PicoMite-XXXX. Accept the "Terminal would like to use Bluetooth" prompt the first time ble_term runs.

Run

```
python ble_term.py PicoMite-77BA
```

(or just python ble_term.py for the default Pico-RP2350 name.)

You'll see a banner with the available commands, then ble_term scans for the device, connects, and you get the MMBasic prompt directly in the same window.

In-terminal commands

ble_term uses Ctrl-] (Telnet-style escape) as its command prefix -- press it once, then a letter:

Key	Command
q, x	Quit ble_term
s	XMODEM send a host file to the Pico
r	XMODEM receive a file from the Pico to the host
c	Clear the host terminal screen (and scrollback)
?, h	Show command help

Any other key after Ctrl-] cancels the command and returns to normal mode. Ctrl-C alone is forwarded to MMBasic as the BASIC break key.

XMODEM file transfer

ble_term implements XMODEM-CRC (128-byte packets) directly over the BLE link -- no external file-transfer tool required. The workflow mirrors the Tera Term style but driven from the one window:

Send a file to the Pico:

1. At the MMBasic prompt, type:

```
XMODEM R progname
```

The Pico starts emitting NAKs (you'll see them in the terminal).

2. Press Ctrl-] then s. ble_term prompts:

```
[xmodem-send] host file to send:
```

3. Type the host path (Backspace and ESC work; ESC cancels), press Enter.
4. Progress is shown as [xmodem] sent N/N bytes, then [xmodem] send complete. Keyboard input is ignored during the transfer.

Receive a file from the Pico:

1. At the MMBasic prompt, type:

```
XMODEM S progname
```

The Pico starts sending packets.

2. Press Ctrl-] then r. ble_term prompts for the host file to save into.
3. Type the host path, press Enter. Progress as [xmodem] received N bytes, then [xmodem] receive complete.

The transfer uses the same chunked-write/retry logic as the bridge, so it works through transient BLE hiccups.

Auto-reconnect

If the Pico reboots (e.g. OPTION CPU_SPEED change or watchdog), ble_term detects the BLE disconnect, scans, and reconnects automatically. Keystrokes typed during the reconnect window are buffered and delivered as soon as the link is back. Each session prints a numbered status line on stderr so you can see what's happening:

```
[ble_term] #1 connected, notify subscribed
...
[ble_term] #1 disconnected (received 482 notifies this session)
[ble_term] session #2 scanning for 'PicoMite-77BA'...
[ble_term] #2 connecting to 28:CD:C1:FF:B7:8D
[ble_term] #2 connected, notify subscribed
```

Key translation

ble_term translates a few host-terminal keystrokes to the forms MMBasic's MMInkey parser recognises:

Host key	Sent to Pico	Effect
Backspace (host sends 0x7F)	0x08 (BS)	Deletes previous char
Home (\033[H or \033OH)	\033[1~	Cursor to start of line
End (\033[F or \033OF)	\033[4~	Cursor to end of line

Arrow keys, Delete, Insert, PgUp/PgDn, F1-F12 already match what MMBasic expects, so they pass through untouched. If you find a key that doesn't behave, the fix is to add a mapping in translate_keys() in ble_term.py.

Alternative -- ble_bridge.py with an external terminal

If you prefer using your existing terminal (Tera Term on Windows, PuTTY on Linux, telnet on macOS) instead of ble_term, ble_bridge.py forwards BLE NUS over TCP and you connect a Telnet client to 127.0.0.1:5555. The bridge

speaks full Telnet NVT (IAC doubling, CR-NUL stuffing) so XMODEM and YMODEM work the same as in WEB build.

The detailed per-OS setup follows. If you're new here, use ble_term above -- it's simpler and covers all three OSes from one script.

Windows (Tera Term)

Windows doesn't expose a BLE serial port as a COM port, so we run a small Python bridge that converts BLE NUS into a TCP socket, then point Tera Term at that socket.

One-off setup

1. Pair the device once via Windows Settings:
- Settings -> Bluetooth & devices -> Add device -> Bluetooth
- Pick PicoMite-XXXX from the list, wait for "Paired".
- Windows will show it as "Not connected" -- that is expected; the Settings UI only handles pairing, not the GATT data stream.

2. Install Python and bleak:
```
pip install bleak
```

3. Configure Tera Term (one-off; Setup -> Save setup... persists these):
- Setup -> Terminal -> Local echo: OFF
- Setup -> Terminal -> Local Edit Mode: OFF
- Setup -> Terminal -> Terminal ID: VT100 (or VT-AT386)

Every session

1. Start the bridge:
```
python ble_bridge.py PicoMite-XXXX
```

The bridge scans, connects, then listens on TCP port 5555: ` [TCP] listening on telnet://127.0.0.1:5555 [BLE] target device: 'PicoMite-XXXX' [BLE] scanning for 'PicoMite-XXXX'... [BLE] found 28:CD:C1:..., connecting... [BLE] connected, subscribing to TX notifications `

2. In Tera Term: File -> New connection -> TCP/IP
- Host: 127.0.0.1
- Service: Telnet
- TCP port#: 5555

3. Hit Enter, you get the MMBasic prompt.

Bridge options

```
python ble_bridge.py                       # default device name "Pico-RP2350"
python ble_bridge.py PicoMite-77BA         # specific device
python ble_bridge.py PicoMite-77BA --port 6000
```

Auto-reconnect

If you reboot the Pico (OPTION CPU_SPEED, error trap, manual reset) the bridge automatically reconnects within ~10 seconds without dropping the Tera Term TCP session. You'll see this in the bridge console:

```
[BLE] disconnected
[BLE] scanning for 'PicoMite-77BA'...
[BLE] found ..., connecting...
[BLE] connected, subscribing to TX notifications
```

Tera Term stays put; data flow pauses then resumes.

Linux (PuTTY)

Same architecture as Windows: pair once, then run ble_bridge.py and point a terminal at TCP 127.0.0.1:5555.

One-off setup

1. Install bleak and PuTTY:

```
pip install bleak
sudo apt install putty            # or: dnf install putty
```

2. Pair the device once via bluetoothctl:

```
$ bluetoothctl
[bluetooth]# scan on
[NEW] Device 28:CD:C1:FF:B7:8D PicoMite-77BA
[bluetooth]# pair 28:CD:C1:FF:B7:8D
[agent] Confirm passkey ... (yes/no): yes
[bluetooth]# trust 28:CD:C1:FF:B7:8D
[bluetooth]# scan off
[bluetooth]# exit
```

The bond is stored in /var/lib/bluetooth/<adapter>/<peer-mac>/.

3. User permissions -- your user must be allowed to use BLE:

```
sudo usermod -aG bluetooth $USER
```

Log out and back in. (Some distros handle this via polkit; if you see DBusError: Not authorized from bleak, this is what to fix.)

Every session

1. Start the bridge:

```
python ble_bridge.py PicoMite-77BA
```

2. Open PuTTY:

- Session -> Connection type: Telnet
- Host Name: 127.0.0.1
- Port: 5555
- Open

Telnet mode is required (Raw mode does NOT work) because the bridge speaks standard NVT Telnet: it negotiates char-at-a-time + server echo via IAC options, doubles 0xFF data bytes as IAC IAC, and stuffs 0x0D (CR) data bytes as CR NUL. PuTTY's Telnet code handles all of that correctly; Raw mode would expose the NUL stuffing and IAC bytes as

garbage characters and break XMODEM.

3. Set vt100 emulation: in the PuTTY config tree, Terminal -> Keyboard leave defaults; Window -> Translation -> Remote character set: UTF-8; Window -> Colours -> ANSI Colour: OK.
4. Save the session (give it a name, click "Save") so you don't have to re-enter the settings.

---

macOS (telnet)

Same architecture as Windows and Linux: pair once, then run ble_bridge.py and connect a Telnet terminal to TCP 127.0.0.1:5555.

One-off setup

1. Install bleak and a Telnet client. macOS removed the built-in telnet binary in 10.15+, so install one via Homebrew:

```
pip3 install bleak
brew install telnet            # or: brew install inetutils
```

(Both packages provide a telnet command.)

2. Pair the device once via the GUI:

- Open System Settings -> Bluetooth.
- Wait for PicoMite-XXXX to appear in the list of nearby devices.
- Click Connect next to it. macOS auto-pairs BLE devices using Just Works -- no prompt for a PIN.
- The device should then show as Connected briefly and drop to Not Connected once macOS finishes pairing. That's expected; the GUI doesn't keep an active GATT session.

3. bleak permissions. Modern macOS prompts for Bluetooth access the first time bleak is used. When you run ble_bridge.py, look for a "Terminal would like to use Bluetooth" alert and accept it. You can also pre-authorise under System Settings -> Privacy & Security -> Bluetooth -> Terminal (or your Python interpreter).

Every session

1. Start the bridge:

```
python3 ble_bridge.py PicoMite-77BA
```

2. In another Terminal window:

```
telnet 127.0.0.1 5555
```

telnet on macOS handles NVT options out of the box, so char-at-a-time mode and CR-NUL stuffing both Just Work -- same reasoning as Windows Tera Term and Linux PuTTY in Telnet mode.

If you'd rather use a graphical terminal, iTerm2 doesn't have a built-in Telnet client, but you can run telnet 127.0.0.1 5555 inside it. CoolTerm and SerialTools are serial-only and won't work without further glue.

3. To exit telnet, type Ctrl-] then quit.

Do not use nc (netcat) -- it's raw TCP and doesn't understand Telnet NVT negotiation, so the keyboard ends up line-buffered and XMODEM transfers break on CR-NUL stuffing.

PicoMite Bluetooth Versions

Pairing model and key storage

PicoMiteBT uses BLE Just Works pairing -- no PIN, no MITM protection. Pairing is automatic; the user doesn't have to type or confirm anything beyond the OS-level "Pair?" dialog.

After pairing, the long-term key (LTK) is stored on the Pico inside the PicoMite Option struct (Option.bt_tlv[] -- a 2 KB area split into two 1 KB banks). It survives a reboot or a power cycle. OPTION RESET, or a firmware update that resets the options, wipes it, and every host then has to pair again.

Up to 4 hosts can be paired at the same time. The Pico keeps a key for each. So you can pair the same Pico with your desk PC, laptop, and phone, and reconnect from whichever one is convenient without re-pairing.

When the table is full and a 5th host pairs, btstack evicts the oldest entry; that host then has to re-pair next time.

Backup serial console

PicoMiteBT has no USB console, so if the Bluetooth link can't be used (no BLE host to hand, a bond mismatch, a radio fault) a UART stands in. By default - after a fresh install or any OPTION RESET - the console also runs on GP8 (TX) and GP9 (RX), UART1 at the console baud rate, alongside the BLE one. Connect a USB-serial adapter there. On a PicoComputer 3 those pins are wired to the board's own USB-serial bridge.

OPTION LIST shows it as OPTION SERIAL CONSOLE COM2,GP8,GP9,BOTH. To move it to other UART pins:

```
OPTION SERIAL CONSOLE GP0, GP1, B
```

Keep the B. On this build it means "Bluetooth and serial": console output goes to both, and either can type. Without it the console is serial only and the BLE console goes silent. To free the pins and use Bluetooth alone:

```
OPTION SERIAL CONSOLE DISABLE
```

Limitations

These are constraints of the design, not bugs to be reported.

One active connection at a time

BLE peripherals advertise to many scanners but accept only one active connection. So while you can be paired with PC + phone + laptop at once, only one of those can have an open session at any moment. The others can't connect until the current one disconnects.

No virtual COM port on Windows

BLE NUS is a GATT profile, not Bluetooth Classic SPP. Windows does not auto-create a COM port for it, and no terminal app talks BLE NUS natively. That's why the Python bridge exists -- it presents a TCP socket that Tera Term/PuTTY can attach to.

If you really want a COMxx device for compatibility with other software, you'd need a virtual-COM driver like com0com and a small custom helper. The bridge could be extended to drive com0com instead of (or in addition to) TCP. Not included.

PicoMite Bluetooth Versions

iOS pairing nuance

iOS will not allow BLE pairing without an active GATT operation following soon after. If you "pair" without then connecting, iOS may silently un-pair. Always connect from an app right after pairing.

Reconnect delay

Rebooting the Pico (OPTION CPU_SPEED, watchdog reset, etc.) means ~10 seconds before the bridge reconnects. Breakdown of where the time goes:

- Pico firmware boot + cyw43 BT firmware download: ~3-5 s
- HCI bring-up + first advertisement: ~1 s
- Host's BLE scan latency: ~2-5 s
- Bridge retry backoff: 1 s

Nothing to do about the Pico-side cost; the bridge could be extended to cache the BD_ADDR and reconnect by address (skipping the scan) for ~3-5 s savings.

Bond mismatch after firmware rebuild

A firmware build that bumps MagicKey triggers ResetOptions() on first boot, which zeroes Option.bt_tlv. The Pico forgets its bonds but the host doesn't. Result on reconnect:

- Windows: characteristic reads return "Unreachable" until you Settings -> Remove device -> re-pair.
- Linux: "Authentication failed" -- fix is bluetoothctl remove <MAC> then re-pair.
- Phone: Forget the device in Bluetooth settings, re-pair.

This only happens after MagicKey changes (i.e., development builds that change the Option layout). Stable releases don't disturb existing bonds.

No persistence on PSRAM-only boards

The bond storage lives in the PicoMite Option flash sector. Boards where PicoMite runs from PSRAM still write Options to flash, so this is not a real limitation -- mentioned only because it's a common question.

Throughput

RFCOMM-class throughput is not the goal -- this is a console link. Typical sustained throughput is ~5-20 KB/s depending on the host's BLE stack and connection parameters. Fast enough for typing, listing programs, and both XMODEM and YMODEM transfers (YMODEM in particular is a strict 8-bit-clean test of the whole pipeline and runs through cleanly at 200 MHz CPU). Not suited for moving large binary files end-to-end.

CPU speed bounds

The cyw43 wireless driver and btstack are polled from the main loop, as the WebMite polls its WiFi. At low CPU speeds, the per-event processing margin becomes too tight for sustained bidirectional traffic and the link destabilises. To prevent this the build enforces a CPU-speed range:

- MIN_CPU = 200 MHz -- the safe floor for BLE on RP2350. Below this the BLE stack can't reliably keep up with AutoSave paste-ins or XMODEM/YMODEM transfers.
- MAX_CPU = 396 MHz -- comfortable overclock headroom for RP2350-A on Pico 2 W. Lower than the standard PicoMite ceiling so the cyw43 SPI link stays well within its 50 MHz rating even when the PIO divider scaling rounds up.

ResetOptions (triggered on first boot after flashing, or after any MagicKey change) sets Option.CPU_Speed to 200 MHz. You can adjust within bounds via OPTION CPU but the firmware rejects values outside [200000, 396000].

PicoMite Bluetooth Versions

When pasting very large programs over BT, prefer AUTOSAVE N (no-echo). The echo path roughly doubles BLE traffic; suppressing it halves the Pico's per-byte workload and tightens timing margins.

Range

Practical ranges to expect, governed by the CYW43439 radio (Class 2 BLE, ~+4 dBm TX, ~-93 dBm RX) and the small PCB antenna on Pico 2 W:

Scenario	Realistic range
Open air, line of sight	20-30 m
Same room with normal furniture	10-15 m
Through one interior wall (drywall)	5-10 m
Through two walls or one concrete wall	2-5 m, often unreliable
Between floors	3-7 m, marginal

Things that improve range:

- Class 1 USB Bluetooth dongle on the host (up to +20 dBm) -- adds several meters, especially through obstructions, because the better receiver on the host side carries the weak Pico transmissions.
- Line of sight. One drywall costs ~10 dB, two walls ~20 dB -- that's most of the link budget. A hallway goes much further than the same distance through walls.
- Antenna orientation. The PCB antenna has lobes; rotating the board 90° can rescue a borderline link. Standing up beats flat on a desk.

Things that don't help much in practice:

- BLE 2M PHY (higher throughput, lower range -- we use 1M which is better balanced for this use).
- BLE Coded PHY (long-range Bluetooth 5) -- neither btstack here nor Windows BLE negotiate it readily for NUS; not configured.
- External antenna mod -- the antenna pad/disable resistor exists on Pico 2 W (like Pico W), but the mod is fiddly and only adds a few dB.

In short: plenty of range for working at a desk, debugging from across the bench, or running BASIC from a phone in the same room -- but don't expect to control devices reliably from another floor of the building.

BLE only -- no Bluetooth Classic

The build links pico_btstack_ble, not pico_btstack_classic. There is no SPP, no audio (A2DP), no HID. Pure BLE GATT for the NUS profile.

Console output buffering during pre-pairing

Anything PicoMite prints before a host has subscribed to TX notifications is buffered in a 1 KB ring on the device. If the device prints more than 1 KB before you connect, the oldest bytes are dropped to make room for newer ones -- so the most recent output (the prompt) is what reaches you when you finally connect.

Troubleshooting

"Could not start notify on 000A: Unreachable" (bleak / Windows)

The host has a cached LTK that doesn't match the Pico's. Remove the device from Windows Settings -> Bluetooth & devices, then pair again.

Bridge keeps cycling connect/disconnect

Same root cause as above. Each cycle is the host attempting reconnect with a stale LTK. Re-pair fixes it.

"Authentication failed" (Linux)

bluetoothctl remove <PicoMAC> then re-pair (scan on, pair, trust).

"DBusError: Not authorized" (Linux)

User isn't in the bluetooth group. sudo usermod -aG bluetooth $USER then log out / back in.

Tera Term shows weird block characters / nothing

- Setup -> Terminal -> Local Edit Mode must be OFF.
- Service must be Telnet (the bridge sends IAC negotiation that Tera Term needs to switch to char-at-a-time mode).
- Local echo must be OFF (PicoMite echoes; turning local echo on doubles every keystroke).

Characters only echo when I press Enter

Tera Term Local Edit Mode is on. Turn it off; save setup.

Connecting from a fresh laptop, BLE LE Explorer shows the device

but characteristics are "Unreachable" The device needs to be paired through OS settings first. BLE Explorer itself cannot drive pairing on Windows; it only browses. Pair via Settings -> Add device, then re-open Explorer.

Bridge prints "[BLE] not found, will retry"

Pico may have rebooted; wait ~10 s. If it persists, check that the device is still advertising -- connect from a phone to verify, or re-flash the firmware. Could also be Windows BLE permissions / adapter state: in Settings -> Bluetooth, toggle Bluetooth off then on to reset the host stack.

"no such file or directory" from nmake in CMake build

Not BLE-related, but a common gotcha: Visual Studio's MSVC tools aren't on PATH. Launch VS Code from the Developer Command Prompt for VS, or CMake: Select a Kit and choose the Visual Studio kit.

Bluetooth host versions: PicoMiteRP2350BTH and PicoMiteHDMIBTH

The other two Bluetooth builds work the other way round. Instead of being a Bluetooth device that a computer connects to, the PicoMite is the Bluetooth host, and keyboards, mice, game controllers, speakers and serial devices connect to it.

- PicoMiteRP2350BTH is the PicoMite RP2350 build (LCD panels, touch, the GUI controls) with Bluetooth added. The console stays on USB. It needs a Raspberry Pi Pico 2 W.
- PicoMiteHDMIBTH is the PicoMiteHDMIUSB build (HDMI video, and USB keyboards, mice, game controllers and touch screens) with the same Bluetooth added. It needs an RP2350 module with the Raspberry Pi wireless chip, such as a Pico 2 W. It was released for the first time in V7.0.00 beta 7.

Everything below applies to both, unless it says otherwise.

PicoMite Bluetooth Versions

The two builds side by side

	PicoMiteRP2350BTH	PicoMiteHDMIBTH
Video	LCD panels	HDMI, 8-bit colour: 640x480 (default), 720x400, 800x600, 848x480, 800x480 or 1024x600
Console	USB	the HDMI screen and keyboard; serial on GP8 (Tx) and GP9 (Rx) as on the other USB versions
Keyboards	Bluetooth LE and Bluetooth Classic	Bluetooth LE, Bluetooth Classic and USB
Mice	Bluetooth LE, a Classic keyboard's touchpad, and PS/2	Bluetooth LE, a Classic keyboard's touchpad, and USB
Audio	a Bluetooth speaker or an I2S DAC	a Bluetooth speaker or an I2S DAC
Serial ports	COM1, COM2, and a Bluetooth serial device as COM7	COM1, COM2, USB serial devices as COM3 to COM6, and a Bluetooth serial device as COM7
CPU speed	200 to 396 MHz, set with OPTION CPUSPEED	set by the HDMI resolution
Largest program	208 KB	180 KB
MMBasic heap	320 KB	220 KB

Keyboards, mice and game controllers

Keyboards, mice and game controllers that use Bluetooth LE (HID over GATT) pair by themselves, as described here. A Bluetooth Classic keyboard, the older kind, is paired with BLUETOOTH CONNECT instead (see Bluetooth Classic keyboards below). A Bluetooth LE keyboard, a Classic keyboard, a speaker and a serial device can all be connected at once.

To pair a Bluetooth LE device, put it into its pairing mode close to the PicoMite. It is found, paired and connected with nothing to type, and the console shows Bluetooth Keyboard Connected. Only devices that are close (roughly in the same room) are paired, so a neighbour's keyboard is left alone.

A few keyboards ask for a passkey while pairing. The PicoMite then shows a six-digit number on the console, for example Bluetooth keyboard pairing: type 123456 on the keyboard being paired, then Enter. Type the number on that keyboard, not on one already connected, and press Enter within about 30 seconds. Nothing appears on the screen while you type it.

A paired keyboard connects again by itself after a reset or a power cycle when a key is pressed. It does not need its pairing button again.

A connected keyboard behaves like a USB or PS/2 one: at the command prompt, in the full screen editor, and in a program with INKEY$ and KEYDOWN(). Set its layout with OPTION KEYBOARD and its repeat timing with OPTION KEYBOARD REPEAT. A Bluetooth mouse is read with DEVICE(MOUSE ...) and can be used with the GUI controls. A game controller is read with DEVICE(GAMEPAD ...).

BLUETOOTH STATUS shows whether a keyboard is connected, and MM.INFO$(BLUETOOTH KEYBOARD) returns its address (or an empty string).

Compact keyboards often leave keys out. The Logitech Pebble Keys 2 K380s, for example, gives F1 to F12 only with Fn held (its top row is media keys), Home, End, Page Up and Page Down only as Fn with an arrow key, and has no Caps Lock light. Media keys work on keyboards that send them in the usual way; the K380s's are not decoded.

PicoMite Bluetooth Versions

Bluetooth Classic keyboards

A Bluetooth Classic keyboard, and the touchpad or mouse built into one, is found with a scan and paired with BLUETOOTH CONNECT, like a speaker. Put the keyboard into its pairing mode, then scan; the keyboard is listed as a Keyboard:

```
> BLUETOOTH SCAN
Scanning...
54:46:6E:AD:87:50  -65 dBm  Keyboard  "Bluetooth 3.0 Keyboard"
1 device found
```

Connect to it by that name or by its address:

```
BLUETOOTH CONNECT "Bluetooth 3.0 Keyboard"
```

If the keyboard asks for a code, the console shows a six-digit number, as for a Bluetooth LE keyboard: type it on the keyboard and press Enter. The command waits up to 60 seconds for that, and prints Connected to keyboard and the address when the keyboard is paired.

The keyboard is then remembered, and connects again by itself whenever it wakes up or is switched on, also after the PicoMite restarts. One Classic keyboard is remembered at a time: connecting another replaces it. Only the remembered keyboard is let in, so another Classic keyboard nearby cannot type into the PicoMite.

Its keys work as a Bluetooth LE keyboard's do, its Caps, Num and Scroll Lock LEDs follow the lock keys, and its touchpad or mouse is read with DEVICE(MOUSE 2, ...), the same as a Bluetooth LE mouse.

Many low-cost keyboards sold for tablets have an operating-system mode, chosen with an Fn key combination or a switch (see the keyboard's instructions). Use its Windows or PC mode. In its iOS mode such a keyboard sends Cmd and an arrow instead of Home, End, Page Up and Page Down, iPad shortcuts instead of F1 to F12, and its touchpad in a different form.

BLUETOOTH STATUS has a line for the Classic keyboard: its address, whether it is connected, and which of its reports carry the keys, the pointer and the media keys (taken from the keyboard when it is paired). When no Bluetooth LE keyboard is connected, MM.INFO$(BLUETOOTH KEYBOARD) returns the Classic keyboard's address.

Speakers and headphones

Anything that supports the A2DP profile will play, which is almost every Bluetooth speaker. Everything PLAY can produce (tones, sounds, WAV, FLAC, MP3 and MOD files, arrays, samples and BBC sounds) goes to the speaker, and a Bluetooth keyboard or mouse keeps working at the same time.

First make Bluetooth the audio output. The option is saved, so this is needed only once, and the PicoMite restarts:

```
OPTION AUDIO BLUETOOTH
```

Then put the speaker into its pairing mode and search for it. The scan lists the address, signal strength and name of each device found, and marks the audio devices and the keyboards. It takes 10 seconds unless you give it a time (2 to 60 seconds):

```
> BLUETOOTH SCAN
```

Connect to the speaker by the name the scan showed (upper or lower case) or by its address. The command waits up to 30 seconds for the speaker and then shows the format of the audio stream:

```
BLUETOOTH CONNECT "BT Speaker"
```

The speaker is then paired and remembered. It is connected again automatically when the PicoMite starts and

whenever the speaker is switched on or comes back into range (the PicoMite keeps trying, at intervals of up to a minute). BLUETOOTH DISCONNECT lets it go until the next BLUETOOTH CONNECT or restart. One speaker can be connected at a time.

If BLUETOOTH CONNECT reports Connection failed (Bluetooth error 0x04), the speaker did not answer in time. This is common just after a speaker is switched on, or while it is still trying to reconnect to the phone or computer it was last used with. Try again.

To save power the stream to the speaker is suspended when nothing has been played for 60 seconds, and restarted by the next sound (which is then delayed by about half a second). Change the time by adding it to the option, from 0 to 250 seconds. 0 keeps the stream running for as long as a speaker is connected:

```
OPTION AUDIO BLUETOOTH 120
```

Some things to expect:

- Bluetooth speakers buffer the sound, so it is heard a fraction of a second (typically 0.2 seconds) after it is played. Music is not affected, but sound effects in a game lag slightly behind the action.
- Encoding the sound takes 12 to 14% of the processor's time at 200 MHz while the stream is running.
- While the stream is running the PicoMite listens less often for keyboards and mice, so a keyboard that has gone to sleep may take a little longer to reconnect.
- PLAY VOLUME scales the sound before it is sent. The speaker's own volume buttons work as usual; the PicoMite does not set the speaker's volume over Bluetooth.

MM.INFO$(BLUETOOTH SPEAKER) returns the address of the connected speaker, or an empty string.

Serial devices (COM7)

A serial device can be connected over Bluetooth and used as the serial port COM7, much as a USB serial device is COM3 to COM6. It can be:

- a Bluetooth Classic device with a serial port (the Serial Port Profile), such as a Bluetooth GPS receiver, or
- a Bluetooth LE device with a serial service: the Nordic UART Service (many modules, and the console of a PicoMiteRP2350BT), an HM-10 module's FFE0 service, or Microchip's Transparent UART.

Make the device discoverable and scan. A Bluetooth LE serial device is listed as BLE serial; a Classic one is listed without a type:

```
> BLUETOOTH SCAN
Scanning...
28:CD:C1:1A:A7:D3  -59 dBm  BLE serial  "PicoMite-500D"
00:19:01:2D:DC:BF  -81 dBm              "XGPS150-2DDCBF"
2 devices found
```

Connect to it by that name or by its address:

```
> BLUETOOTH CONNECT "XGPS150-2DDCBF"
Connected to serial device 00:19:01:2D:DC:BF (RFCOMM channel 1) as COM7
```

For a Classic device the PicoMite first asks it for its serial port (one without a serial port is tried as a speaker), then pairs with it, answering a PIN request with 0000. A Bluetooth LE device is used without pairing unless it asks for it. The device is then remembered. One serial device is remembered at a time: connecting another replaces it.

After that, COM7 is used like any other serial port:

```
OPEN "COM7:" AS #1
OPEN "COM7:" AS GPS
```

A baud rate may be given but is ignored, and so are 7BIT, EVEN, ODD, S2 and INV. A buffer size, an interrupt and its level work as they do for COM1. PRINT # ends each line with CR and LF.

While COM7 is open, the remembered device is reconnected whenever it is not connected: when the port is opened, after the device is switched off and on, and when it comes back into range. A Classic device is retried at intervals of up to a minute, a Bluetooth LE device as soon as it advertises. When the device comes or goes by itself, the console shows Bluetooth serial device connected (COM7) or Bluetooth serial device disconnected (COM7). CLOSE leaves the device connected, and a closed port does not reconnect it.

To let the device go, use BLUETOOTH DISCONNECT SERIAL: it is not reconnected until COM7 is next opened or the next BLUETOOTH CONNECT. BLUETOOTH STATUS has a Serial: line with the bytes received and sent, and MM.INFO$(BLUETOOTH SERIAL) returns the connected device's address, or an empty string, so a program can wait for it.

Things to know:

- Some GPS receivers have a switch for use with an iPhone or iPad that hides their serial port. Set it for other devices; on the Dual XGPS150 that is the position towards the USB connector.
- A Bluetooth LE device that is switched off is noticed only when the link times out, which can take several seconds (16 seconds with a PicoMiteRP2350BT, which asks for a 10-second timeout).
- Older GPS receivers can report a date 1024 weeks (19.6 years) too early, a fault in their own firmware known as the GPS week rollover. The XGPS150 is one of them.

Pairings

The pairings of up to four Bluetooth LE devices (keyboards, mice, game controllers and serial devices), and of up to four Classic speakers, keyboards and serial devices together, are stored with the saved options, so they survive a power cycle, and so are the remembered speaker, Classic keyboard and serial device. BLUETOOTH FORGET deletes all of them, and BLUETOOTH FORGET addr$ only those of one device (a connected device that is forgotten is disconnected). OPTION RESET, and a firmware update that resets the options, delete them too. The devices then have to be paired again.

BLUETOOTH STATUS lists the connections first. The lines after them are diagnostics, such as the number of keyboard pairings held and how the last pairing went (for example "interrupted, the keyboard dropped the link"), which are useful when reporting a problem.

What the host builds leave out

- PWM audio, the MCP48n2 SPI DAC and the VS1053 (with the PLAY commands that need it, such as PLAY MIDI and PLAY STREAM). The audio outputs are Bluetooth and an I2S DAC.
- PicoMiteRP2350BTH: a PS/2 keyboard (a PS/2 mouse still works).
- PicoMiteHDMIBTH: PS/2 devices, the CAMERA commands and YMODEM, and the full set of HDMI resolutions (it has the same reduced set as the PicoMiteHDMIWEB).
- WiFi, on both.
- PIO2, which the wireless chip uses (an I2S DAC shares it). PIO0 and PIO1 remain for programs.

Boards that wire the wireless chip differently

All three Bluetooth builds expect the wireless chip on the same pins as the Pico 2 W: GP23 (power on), GP24 (data), GP25 (chip select) and GP29 (clock). A module that uses other pins needs OPTION CYW43 PINS, which lists them in that order. For example, the Waveshare RP2350B Pico WiFi board needs:

```
OPTION CYW43 PINS GP36, GP37, GP38, GP39
```

PicoMite Bluetooth Versions

The pins given are reserved instead of GP23, GP24, GP25 and GP29, which become normal I/O pins. If the wireless chip does not answer on the pins set, the PicoMite says CYW43 radio failed to start on ... when it starts and runs without Bluetooth. OPTION CYW43 PINS DEFAULT, or OPTION RESET, goes back to the standard pins. If an I2S DAC is used, it shares a PIO with the wireless chip, so its pins and the data and clock pins must all be within GP0 to GP31 or all within GP16 to GP47.

Files of interest in this build

File	Purpose
BTConsole.c / BTConsole.h	BLE NUS server, TX/RX rings, GATT/SM hooks
nus_gatt.gatt	GATT database (NUS UUIDs, characteristics, flags); compiled to nus_gatt.h by btstack's compile_gatt.py
btstack_config.h	btstack feature flags; controls bonding, encryption, max peer count
ble_term.py	Cross-platform self-contained BLE NUS terminal with built-in XMODEM (recommended desktop path)
ble_bridge.py	Alternative cross-platform BLE NUS -> TCP forwarder for users who prefer an external Telnet client
FileIO.h	Defines Option.bt_tlv[2048] for persistent bond storage
configuration.h	PICOMITEBT block: flash offsets, magic key, heap sizes
CMakeLists.txt	PICOBTRP2350 variant definition; links pico_btstack_ble, pico_btstack_cyw43, pico_cyw43_arch_poll (the host builds PICOBTHRP2350 and HDMIBTH also compile the Bluetooth Classic sources that A2DP needs)
BTKeyboard.c / BTKeyboard.h	Host builds: the BLE HID host - scanning, pairing, keyboards, mice and game controllers
BTAudio.c / BTAudio.h	Host builds: the A2DP source for speakers and the BLUETOOTH command
BTClassicKeyboard.c / BTClassicKeyboard.h	Host builds: Bluetooth Classic keyboards and their touchpads (btstack's hid_host, report protocol, the layout read from the HID descriptor)
BTSerial.c / BTSerial.h	Host builds: a Bluetooth serial device as COM7 - Classic (SDP lookup, RFCOMM) or BLE (Nordic UART, HM-10, Microchip Transparent UART over GATT)